# Faster Symbolic Music Generation via Cross-Domain Transfer of Autoregressive Inference-Acceleration Techniques

*A Comparative Study of Speculative Decoding and Medusa Decoding Applied to Music Generation*

James Gough

COMP 3106 — Project Proposal

## 1 Background and Objectives

Modern autoregressive models generate one output token at a time, which is intrinsically latency bound: each and every token requires a full forward pass, and passes cannot be parallelized because each depends on the last one. Two inference-acceleration techniques address this without affecting the model's output distribution: (1) **speculative decoding**, in which a small "draft" model proposes several tokens ahead and a larger "target" model verifies them, all in one pass, and (2) **Medusa decoding**, where extra prediction heads are attached to the target model architecture itself to suggest several tokens in advance. Both methods exploit how single-token decoding forgoes parallelization, and can reduce inference latency on sequential generation tasks substantially.

These techniques have been applied across a growing range of autoregressive contexts, but neither have been extended to music. Moreover, there have been no direct comparisons between the two techniques outside language generation. This project investigates whether they transfer to a structurally different sequential domain, symbolic music generation, and whether their performance characteristics are consistent with those that have been seen with language. There are two objectives: (a) verify both method implementations preserve the output quality of a baseline autoregressive transformer model, and (b) compare the speedup in generation latency each achieves over the baseline, identifying which one yields greater acceleration.

## 2 Proposed Method(s) from Artificial Intelligence

The baseline target model will be a decoder-only transformer (approximately 6 layers, 256-dimensional hidden layer size, around 4-8M parameters) trained via next-token prediction (using cross-entropy loss) over tokenized symbolic music sequences. This will be the same as the architecture and procedure behind training autoregressive language models on text. The two acceleration methods will then be layered onto this same frozen model:

- **Speculative decoding:** a separately trained, substantially smaller draft transformer (approximately 1-2 layers, 64-dimensional hidden layer size, around 25x fewer parameters) proposes multiple tokens ahead (4-5, so the lookahead depth used for both methods match). A proposed token is accepted with the probability $min(1, p/q)$ ($p$, $q$ = target and draft probabilities), and is resampled from the target's residual distribution if rejected, so outputs match sampling from the target model alone.
- **Medusa decoding:** 4-5 additional linear prediction heads are trained on top of the frozen target model, with each head predicting a further future token. The target model walks through a tree-based verification process in parallel, only keeping proposed tokens it would have produced itself to maintain the same output quality.

## 3 Dataset or Environment

The **Nottingham Music Database** containing around 1,200 British and American folk melodies will be used as the project dataset. Each song is encoded in ABC notation that records pitch, duration, and bar structure as characters. As part of this project, each melody will be parsed and converted into a sequence of discrete symbolic tokens (individual notes, durations, and bar-line markers), so the model's inputs at training and inference time are structurally identical to how a language model takes in a sequence of word/subword tokens. The original dataset is sufficiently sized, freely available, and has been extensively used in prior music generation literature, which supports fast iteration within the project timeline and provides known reference points for expected model behavior. Sequences will be split into training, validation, and test partitions at song level to prevent data leakage.

## 4 Proposed Validation/Analysis Strategy

Both **generation quality** and **acceleration effectiveness** will be evaluated against the frozen target model as a common reference. Generation quality will be measured via valid-generation rate (rate of syntactically and structurally valid outputs) and assessed for statistical similarity with the real test set using Jensen–Shannon (JS) divergence on four musical categories: pitch-class, note duration, melodic interval (note-to-note jumps), and repetition rate. Quality of the baseline model will also be assessed via held-out perplexity. For both methods, generated sequences will be compared to the target model's outputs using perplexity and JS divergence to verify that they preserve its sampling distributions, rather than just producing plausible-sounding but distributionally different results. Qualitative evaluation will involve test participants attempting to correctly identify real melodies from those generated by the baseline or the 2 methods. The participants will also be asked to rate generated melodies from 1 to 5 on well-defined criteria like coherence, rhythm, structure, and overall plausibility. Inference acceleration will be measured with time-to-first-token (how long before first output token is produced), average generation time, and time per output token to capture inference speedup. Token acceptance rate and target model forward pass count will also be tracked to explain the underlying computational savings driving the speedup. While both methods are built on top of the same frozen target model, in order to maintain fairness in evaluations too, the prompts, hardware, and lookahead depth (number of tokens generated in advance) will be the same for both techniques. The additional training cost of each method will be reported separately to contextualize observed performances.

## 5 Description of the Novelty

Speculative decoding and Medusa decoding are now established inference-acceleration techniques that have been demonstrated in a few autoregressive domains, including language, code, speech, biological sequences, and video. This variety suggests that these methods exploit a general property of autoregressive generation rather than something specific just to text. By applying both methods to symbolic music, this project will test that claim in a context with considerably different structure to previously studied domains. The two methods have been directly compared against each other for text generation and, more recently, for vision-language generation, but no prior work has been identified that compares speculative decoding and Medusa decoding for symbolic sequence generation outside of language. This project will establish whether, and to what degree, these approaches extend to symbolic music, and provide evidence as to whether their relative advantage is a generic property transferable across domains or specific to language's specific statistical structure.

## 6 Weekly Schedule of Milestones

| Week | Week Of | Milestone |
|---|---|---|
| **1** | Sept 28 | Acquire and preprocess Nottingham dataset (ABC notation); build tokenizer; set up MIDI/audio rendering pipeline (music21 + FluidSynth) end-to-end on a dummy sequence. |
| **2** | Oct 5 | Implement target transformer (baseline model); write evaluation metrics (valid-generation rate, JS divergences, perplexity); assess baseline on held-out data. |
| **3** | Oct 12 | Begin speculative decoding implementation with rejection-sampling verification. |
| **4** | Oct 19 | Finish speculative decoding; begin Medusa implementation. |
| **5** | Oct 26 | Finish implementation of Medusa heads with tree-based verification. |
| **6** | Nov 2 | Run all quantitative quality and speedup tests; begin human listening evaluations. |
| **7** | Nov 9 | Finish human evaluations; build the interactive demo interface (seed selection, live timing, audio playback). |
| **8** | Nov 16 | Analyze results and produce final plots/tables; write Methods and Results report sections. |
| **9** | Nov 23 | Finish Introduction and Discussion; complete repository documentation. |
| **10** | Nov 30 | Buffer week: resolve outstanding issues; polish implementation and demo. |